

Retail Dynamics – Exploring Sales Trends

Capstone Project

Name – Pratik Rokade

Ent id – pratik.r.rokade@accenture.com

Emp id – 14182788

❖ Project Overview -

This document details the end-to-end data pipeline and application development for the RetailMart Group using Palantir Foundry. The project encompasses data ingestion, programmatic and visual transformations, object modeling, and the creation of an interactive operational application.

Datasets Utilized

1. Stores: Contains metadata for RetailMart locations, including Store ID, store Type, and Size (square footage).
2. Features: Contains time-series macroeconomic variables and regional indicators per store, including Temperature, Fuel Price, Markdowns (1-5), CPI, Unemployment, and an IsHoliday boolean.
3. Sales: Transactional historical dataset detailing Weekly Sales aggregated by Store ID, Department, and Date.

❖ Data Transformation using Code Repositories –

The Code Repository in Palantir Foundry was used to develop and manage the programmatic data transformations required for the project. It provides an environment to write transformation code using SQL and Python/PySpark and generate output datasets that can be used in the downstream Pipeline Builder, Ontology, Quiver, and Workshop components

1 SQL –

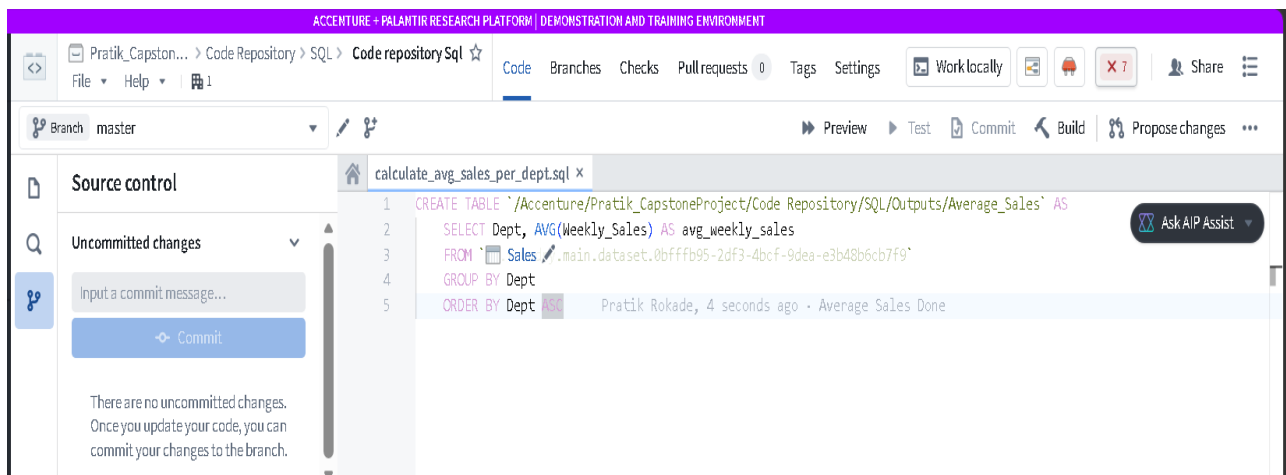
Steps:

1. Load the required Sales, Stores, and Features datasets.
2. Calculate average and total sales using SQL aggregation functions.
3. Group sales by Store, Department, and Month.
4. Join Sales with Stores and Features using common keys.
5. Apply filters to analyze specific conditions such as holidays.
6. Generate the transformed datasets as outputs.

Transformation 1 – Calculate Average Sales per Department Across All Stores :

Generate a summary view that computes AVG(Weekly_Sales) grouped by Dept across all stores for benchmarking department performance.

Code –



The screenshot shows the Palantir Research Platform SQL editor interface. The top bar indicates the environment is 'ACCENTURE + PALANTIR RESEARCH PLATFORM | DEMONSTRATION AND TRAINING ENVIRONMENT'. The breadcrumb navigation shows 'Pratik_Capston...' > 'Code Repository' > 'SQL' > 'Code repositorySql'. The left sidebar shows 'Source control' with 'Uncommitted changes' and a 'Commit' button. The main editor area displays a SQL query in a file named 'calculate_avg_sales_per_dept.sql'. The query is as follows:

```
1 CREATE TABLE `/Accenture/Pratik_CapstoneProject/Code Repository/SQL/Outputs/Average_Sales` AS
2 SELECT Dept, AVG(Weekly_Sales) AS avg_weekly_sales
3 FROM `Sales`
4 GROUP BY Dept
5 ORDER BY Dept
```

The query is executed, and the results are shown at the bottom: 'Pratik Rokade, 4 seconds ago · Average Sales Done'. An 'Ask AIP Assist' button is visible on the right side of the editor.

Output –

	Dept integer	avg_weekly_sales double
1	1	22513.322937062956000
2	2	46102.090419580410000
3	3	13150.478041958038000
4	4	36964.154475524450000
5	5	24257.941118881110000
6	6	4801.780139860141000
7	7	24566.487412587410000
8	8	35718.257622377610000

Build –

NAME	LAST UPDATED	TAGS
Average_Sales	Wed, Aug 26, 2026, 4:18:...	

Transformation 2 – Aggregate Weekly Sales by Store and Month:

Create a summary table that groups Weekly_Sales by Store and extracts the month from Date. Use GROUP BY Store, MONTH(Date) and calculate SUM(Weekly_Sales) for monthly performance tracking.

Code –

```
aggregate_weekly_sales_by_stores_and_month.sql x
1 CREATE TABLE `./Accenture/Pratik_CapstoneProject/Code Repository/SQL/Outputs/Average_Weekly_Sales` AS
2 SELECT
3     Store,
4     Month(TO_DATE(Date, 'dd/MM/yyyy')) AS MONTH,
5     SUM(Weekly_Sales) AS Total_Weekly_Sales
6 FROM `Sales` .main.dataset.0bffffb95-2df3-4bcf-9dea-e3b48b6cb7f9`
7
8 GROUP BY Store,
9     Month(TO_DATE(Date, 'dd/MM/yyyy'))
10
11 ORDER BY MONTH ASC
```

Pratik Rokade, 5 seconds ago · Transform2 Done

Output –

Output Average_Weekly_Sales x					Showing 12 rows	3 columns	Search columns...
	Store 123 integer		MONTH 123 integer			Total_Weekly_Sales 1.0 double	
1		1		1		10979435.660000006	
2		1		2		19203478.829999999	
3		1		3		20050314.849999998	
4		1		4		21307369.009999998	
5		1		5		18218545.909999996	
6		1		6		19975621.549999970	
7		1		7		20411255.670000000	
8		1		8		19684367.939999953	
9		1		9		18855474.380000000	
10		1		10		19015620.219999970	
11		1		11		13095886.350000009	
12		1		12		17586860.390000008	

Build –

Pratik_CapstoneProject ☆ 1

★ Pinned The most important files in this project 0 items

> Code Repository > SQL > Outputs ☆ 1 Actions + New

NAME ^	LAST UPDATED	TAGS
Average_Sales	Wed, Aug 26, 2026, 4:18:...	
Average_Weekly_Sales	Wed, Aug 26, 2026, 4:48:...	

Transformation 3 - Join Sales and Store Attributes for Enhanced Analysis:

Perform an INNER JOIN between Sales and Stores on Store to enrich sales data with store attributes like Type and Size. Create a composite key store_dept_key by concatenating Store and Dept, then calculate the average weekly sales for each store_dept_key to enable department-level performance analysis.

Code -

```
join_sales_and_store_attributes.sql x
1 CREATE TABLE `./Accenture/Pratik_CapstoneProject/Code Repository/SQL/Outputs/Join_Sales_States` AS
2
3 SELECT
4     CONCAT(
5         CAST(s.Store AS STRING),
6         '-',
7         CAST(s.Dept AS STRING)
8     ) AS store_dept_key,
9
10    AVG(s.Weekly_Sales) AS avg_weekly_sales
11
12 FROM `Sales` main.dataset.0bffffb95-2df3-4bcf-9dea-e3b48b6cb7f9` AS s
13
14 INNER JOIN `Stores` main.dataset.bda7c97d-3b21-4978-a5b6-f6a5d4d3ec40` AS st
15
16 ON s.Store = st.Store
17
18 GROUP BY
19     s.Store,
20     s.Dept
21
22 ORDER BY
23     s.Dept DESC 1 minute ago - Uncommitted changes
```

Output -

Output Join_Sales_Stores x			
	store_dept_key string	avg_weekly_sales double	
1	1_97	34414.802278481010000	
2	1_96	33251.831639344250000	
3	1_95	120772.062167832120000	
4	1_94	63180.568181818206000	
5	1_93	71699.182377622330000	
6	1_92	135458.969510489580000	
7	1_91	64238.943426573416000	
8	1_90	82427.547832167870000	
9	1_87	46006.277202797220000	
10	1_85	3685.875524475524300	
11	1_83	5704.048951048958000	
12	1_82	20727.860909090900000	
13	1_81	29780.696573426576000	
14	1_80	17139.314615384614000	
15	1_79	33974.792937062940000	
16	1_78	11.500000000000000	
17	1_77	677.000000000000000	

Build –

Pinned The most important files in this project 0 items

> Code Repository > SQL > Outputs ☆ 1

Actions ▾ + New ▾

NAME ^	LAST UPDATED	TAGS
Average_Sales	Wed, Aug 26, 2026, 4:18:14 PM	
Average_Weekly_Sales	Wed, Aug 26, 2026, 4:48:30 PM	
Join_Sales_Features	Wed, Aug 26, 2026, 5:51:24 PM	
<u>Join_Sales_Stores</u> ☆	Tue, Sep 1, 2026, 3:29:15 PM	

Transform 4 - Join Sales and Features for Holiday Analysis:

Use an INNER JOIN between Sales and Features on Store and Date. Filter records where `is_holiday = TRUE` and calculate total `Weekly_Sales` during holidays grouped by Store.

Code –

```
join_sales_and_features_for_holiday_analysis.sql x
1 CREATE TABLE `./Accenture/Pratik_CapstoneProject/Code Repository/SQL/Outputs/Join_Sales_Features` AS
2
3 SELECT
4     s.Store,
5     SUM(s.Weekly_Sales) AS holiday_sales
6
7 FROM `Sales` main.dataset.0bffffb95-2df3-4bcf-9dea-e3b48b6cb7f9` AS s
8
9 INNER JOIN `Features` main.dataset.2d0a6a5b-6307-46e2-ad96-f16852648083` AS f
10
11 ON s.Store = f.Store
12 AND to_date(s.Date, 'dd/MM/yyyy') =
13     to_date(f.Date, 'dd/MM/yyyy')
14
15 WHERE s.IsHoliday = TRUE
16
17 GROUP BY s.Store
```

Output –

Output			Join_Sales_Features ×	
			Showing 1 row	2 columns
	Store 123 integer	▼	holiday_sales 1.0 double	▼
1		1	16334239.860000014	

Build –

★ Pinned The most important files in this project			0 items
> Code Repository > SQL > Outputs ☆ 1			+ New ▾
NAME ^	LAST UPDATED	TAGS	
📊 Average_Sales	Wed, Aug 26, 2026, 4:18:...		
📊 Average_Weekly_Sales	Wed, Aug 26, 2026, 4:48:...		
📊 Join_Sales_Features ☆	Wed, Aug 26, 2026, 5:51:...		
📊 Join_Sales_States	Wed, Aug 26, 2026, 5:35:...		

❖ Python –

Python transformations were used for data cleansing, standardization, validation, and feature engineering. PySpark was used to efficiently process the datasets and prepare them for downstream analysis.

Steps:

1. Load the required datasets into the transformation.
2. Handle missing values in relevant columns.
3. Rename columns into a standardized format.
4. Validate and convert columns to appropriate data types.
5. Remove unnecessary columns.
6. Create derived columns and composite keys.
7. Generate datasets via build.

Transformation 1 - Handling Missing Values and Create Unique Record ID: Replace NA in Markdown1–Markdown5 with 0 and fill missing CPI and Unemployment with their respective column means. Then, create a new column record_id by concatenating Store and Date to uniquely identify each record in Features.

Code –

```
handling_missing_values.py x
1 from transforms.api import transform_df, Input, Output
2 from pyspark.sql import functions as F
3
4
5 Replace paths with RIDs
6 @transform_df(
7     Output("Handling_Missing_Values", "Project/Code Repository/Python/Output/Handling_Missing_Values"),
8     features_df = Input("Features", "in.dataset.2d0a6a5b-6307-46e2-ad96-f16852648083")
9 )
10 def compute(features_df):
11     # NA/Null to 0
12     markdown_columns = [
13         "Markdown1",
14         "Markdown2",
15         "Markdown3",
16         "Markdown4",
17         "Markdown5"
18     ]
19     for column_name in markdown_columns:
20         features_df = features_df.withColumn(column_name,
21             F.when(
22                 F.col(column_name).isNull() |
23                 (F.trim(F.col(column_name)) == "NA"),
24                 F.lit(0.0)
25             )
26         )
27
28     # CPI to double
29     features_df = features_df.withColumn("CPI",
30         F.when(
31             F.col("CPI").isNull() |
32             (F.trim(F.col("CPI")) == "NA"),
33             F.lit(None).cast("double")
34         ).otherwise(
35             F.col("CPI").cast("double")
36         )
37     )
38
39     # Unemployment to double
40     features_df = features_df.withColumn("Unemployment",
41         F.when(
42             F.col("Unemployment").isNull() |
43             (F.trim(F.col("Unemployment")) == "NA"),
44             F.lit(None).cast("double")
45         ).otherwise(
46             F.col("Unemployment").cast("double")
47         )
48     )
```



```

handling_missing_values.py x
39     features_df = features_df.withColumn("Unemployment",
44         ).otherwise(
45             F.col("Unemployment").cast("double")
46         )
47     )
48     # mean CPI
49     cpi_mean = features_df.select(F.avg("CPI").alias("cpi_mean")).collect()[0]["cpi_mean"]
50
51     # mean Unemployment
52     unemployment_mean = features_df.select(F.avg("Unemployment").alias("unemployment_mean")
53     ).collect()[0]["unemployment_mean"]
54
55     # Fill missing CPI with mean
56     features_df = features_df.withColumn("CPI",
57         F.coalesce(
58             F.col("CPI"),
59             F.lit(cpi_mean)
60         )
61     )
62     # Fill missing Unemployment with mean
63     features_df = features_df.withColumn("Unemployment",
64         F.coalesce(
65             F.col("Unemployment"),
66             F.lit(unemployment_mean)
67         )
68     )
69     # Create unique record ID using Store and Date
70     features_df = features_df.withColumn("record_id",
71         F.concat_ws(
72             "_",
73             F.col("Store").cast("string"),
74             F.date_format(
75                 F.to_date(F.col("Date"), "dd/MM/yyyy"),
76                 "yyyy-MM-dd"
77             )
78         )
79     )
80     return features_df

```

Output –

Output

Handling_Missing_Values

Showing 8,190 rows

13 columns

Search columns...

	Store integer	Date date	Temperature 1.0 double	Fuel_Price 1.0 double	MarkDown1 1.0 double	MarkDown2 1.0 double	MarkDow 1.0 double
1	17	2011-10-14	47.69	3.567	0.00	0.00	
2	17	2011-10-21	47.27	3.579	0.00	0.00	
3	17	2011-10-28	41.89	3.567	0.00	0.00	
4	17	2011-11-04	37.52	3.538	0.00	0.00	
5	17	2011-11-11	27.61	3.513	3935.00	4172.99	
6	17	2011-11-18	32.93	3.489	6004.88	247.29	
7	17	2011-11-25	32.81	3.445	310.16	181.17	
8	17	2011-12-02	26.47	3.389	5562.36	222.94	
9	17	2011-12-09	17.56	3.341	8530.15	0.00	
10	17	2011-12-16	21.07	3.282	3356.86	0.03	
11	17	2011-12-23	18.76	3.186	945.50	35.63	
12	17	2011-12-30	26.73	3.119	4439.21	42391.85	
13	17	2012-01-06	27.64	3.080	5185.45	12090.50	
14	17	2012-01-13	20.28	3.056	1552.86	3911.73	
15	17	2012-01-20	27.26	3.047	1461.59	577.33	
16	17	2012-01-27	29.99	3.058	1834.70	684.22	
17	17	2012-02-03	26.26	3.077	32793.78		
18	17	2012-02-10	23.63	3.116	5658.48		

Calculate row count

Output

Handling_Missing_Values

Showing 8,190 rows

13 columns

Search columns...

	MarkDown4 1.0 double	MarkDown5 1.0 double	CPI 1.0 double	Unemployment 1.0 double	IsHoliday boolean	record_id string
1	0.00	0.00	129.77064520000000	6.6170000000000000	false	17_2011-10-14
2	0.00	0.00	129.78216130000000	6.6170000000000000	false	17_2011-10-21
3	0.00	0.00	129.79367740000000	6.6170000000000000	false	17_2011-10-28
4	0.00	0.00	129.80519350000000	6.6170000000000000	false	17_2011-11-04
5	225.67	2515.61	3009.79	6.6170000000000000	false	17_2011-11-11
6	32.39	653.49	4391.01	6.6170000000000000	false	17_2011-11-18
7	62095.85	106.24	1292.30	6.6170000000000000	true	17_2011-11-25
8	16380.88	2303.32	17707.05	6.6170000000000000	false	17_2011-12-02
9	29.60	840.04	6700.99	6.6170000000000000	false	17_2011-12-09
10	38.78	1286.21	2618.88	6.6170000000000000	false	17_2011-12-16
11	1077.82	57.64	2404.82	6.6170000000000000	false	17_2011-12-23
12	120.08	1366.90	2383.67	6.6170000000000000	true	17_2011-12-30
13	33.90	1490.06	4832.68	6.4030000000000000	false	17_2012-01-06
14	0.52	542.24	1709.97	6.4030000000000000	false	17_2012-01-13
15	0.00	442.39	1407.74	6.4030000000000000	false	17_2012-01-20
16	0.00	118.00	1109.10	6.4030000000000000	false	17_2012-01-27
17	38.96	35785.26	4518.58	6.4030000000000000	false	
18	16.27	4292.59	6008.71	6.4030000000000000	true	

Calculate row count

OverviewTestsFile ChangerBuildDocTask RunnerSQLTerminal

Build -

Pinned

The most important files in this project

0 items

> Code Repository > Python > Output

+ New

NAME	LAST UPDATED	TAGS
Handling_Missing_Values	Wed, Aug 26, 2026, 9:44:...	

Transformation 2 - Renaming Columns and Standardize Format:

Rename columns in Features to snake_case (e.g., Fuel_Price → fuel_price, IsHoliday → is_holiday). After renaming, create a new column store_date_key by combining store and date for uniqueness.

Code –

```
renaming_columns_and_standardize_format.py ×
1 from transforms.api import transform_df, Input, Output
2 from pyspark.sql import functions as F
3
4
5 @transform_df(
6     Output("/Accenture/Pratik_CapstoneProject/Code Repository/Python/Output/Renaming_Columns"),
7     features_df = Input("Features", in_dataset="2d0a6a5b-6307-46e2-ad96-f16852648083")
8 )
9 def compute(features_df):
10
11     features_df = (
12         features_df
13         .withColumnRenamed("Store", "store")
14         .withColumnRenamed("Date", "date")
15         .withColumnRenamed("Temperature", "temperature")
16         .withColumnRenamed("Fuel_Price", "fuel_price")
17         .withColumnRenamed("MarkDown1", "markdown1")
18         .withColumnRenamed("MarkDown2", "markdown2")
19         .withColumnRenamed("MarkDown3", "markdown3")
20         .withColumnRenamed("MarkDown4", "markdown4")
21         .withColumnRenamed("MarkDown5", "markdown5")
22         .withColumnRenamed("CPI", "cpi")
23         .withColumnRenamed("Unemployment", "unemployment")
24         .withColumnRenamed("IsHoliday", "is_holiday")
25     )
```

```
renaming_columns_and_standardize_format.py ×
11     features_df = (
12         features_df
13         .withColumnRenamed("MarkDown5", "markdown5")
14         .withColumnRenamed("CPI", "cpi")
15         .withColumnRenamed("Unemployment", "unemployment")
16         .withColumnRenamed("IsHoliday", "is_holiday")
17     )
18
19     # Create store_date_key
20     features_df = features_df.withColumn(
21         "store_date_key",
22         F.concat_ws(
23             "_",
24             F.col("store").cast("string"),
25             F.date_format(
26                 F.to_date(F.col("date"), "dd/MM/yyyy"),
27                 "yyyy-MM-dd"
28             )
29         )
30     )
31
32     return features_df
```

Output –

Output		Renaming_Columns ×				Showing 8,190 rows		13 columns	Search columns...	
	store integer	date date	temperature double	fuel_price double	markdown1 string	markdown2 string	markdown			
1	17	2011-10-14	47.69	3.567	NA	NA	NA			
2	17	2011-10-21	47.27	3.579	NA	NA	NA			
3	17	2011-10-28	41.89	3.567	NA	NA	NA			
4	17	2011-11-04	37.52	3.538	NA	NA	NA			
5	17	2011-11-11	27.61	3.513	3935	4172.99	225.67			
6	17	2011-11-18	32.93	3.489	6004.88	247.29	32.39			
7	17	2011-11-25	32.81	3.445	310.16	181.17	62095.85			
8	17	2011-12-02	26.47	3.389	5562.36	222.94	16380.88			
9	17	2011-12-09	17.56	3.341	8530.15	NA	29.6			
10	17	2011-12-16	21.07	3.282	3356.86	0.03	38.78			
11	17	2011-12-23	18.76	3.186	945.5	35.63	1077.82			
12	17	2011-12-30	26.73	3.119	4439.21	42391.85	120.08			
13	17	2012-01-06	27.64	3.080	5185.45	12090.5	33.0			
14	17	2012-01-13	20.28	3.056	1552.86	3911.73	0.52			
15	17	2012-01-20	27.26	3.047	1461.59	577.33	NA			
16	17	2012-01-27	29.99	3.058	1834.7	684.22	NA			
17	17	2012-02-03	26.26	3.077	32793.78	5509.47	10.27			
18	17	2012-02-10	23.63	3.116	5658.48	13498.72	10.27			

Output		Renaming_Columns ×				Showing 8,190 rows		13 columns	Search columns...	
	markdown4 string	markdown5 string	cpi string	unemployment string	is_holiday boolean	store_date_key string				
1	NA	NA	129.7706452	6.617	false	17_2011-10-14				
2	NA	NA	129.7821613	6.617	false	17_2011-10-21				
3	NA	NA	129.7936774	6.617	false	17_2011-10-28				
4	NA	NA	129.8051935	6.617	false	17_2011-11-04				
5	2515.61	3009.79	129.8167097	6.617	false	17_2011-11-11				
6	653.49	4391.01	129.8268333	6.617	false	17_2011-11-18				
7	106.24	1292.3	129.8364	6.617	true	17_2011-11-25				
8	2303.32	17707.05	129.8459667	6.617	false	17_2011-12-02				
9	840.04	6700.99	129.8555333	6.617	false	17_2011-12-09				
10	1286.21	2618.88	129.8989645	6.617	false	17_2011-12-16				
11	57.64	2404.82	129.9845484	6.617	false	17_2011-12-23				
12	1366.9	2383.67	130.0710323	6.617	true	17_2011-12-30				
13	1490.06	4832.68	130.1575161	6.403	false	17_2012-01-06				
14	542.24	1709.97	130.244	6.403	false	17_2012-01-13				
15	442.39	1407.74	130.2792258	6.403	false	17_2012-01-20				
16	118	1109.1	130.3144516	6.403	false	17_2012-01-27				
17	35785.26	4518.58	130.3496774	6.403	false	17_2012-02-03				
18	4292.59	6008.71	130.3849932	6.403	true	17_2012-02-10				

Build –

Pinned The most important files in this project			0 items
Code Repository > Python > Output ☆ 1			
+ New			
NAME	LAST UPDATED	TAGS	
Handling_Missing_Values	Wed, Aug 26, 2026, 9:44:...		
Renaming_Columns ☆	Thu, Aug 27, 2026, 1:39:...		

Transformation 3 – Validating and Converting Data Types:

Ensure that numeric fields are stored in the correct data type. Columns like Weekly_Sales, Temperature, Fuel_Price, MarkDown1–5, CPI, and Unemployment should be of type DoubleType for accurate numerical operations.

Code –

```
converting_datatypes.py x
1 from pyspark.sql import functions as F
2 from transforms.api import transform_df, Input, Output
3 from pyspark.sql.types import DoubleType    2 minutes ago · Uncommitted changes
4
5
6 @transform_df(
7     Output("/Accenture/Pratik_CapstoneProject/Code Repository/Python/Output/Datasets/Datatypes"),
8     features_df=Input("Features", in.dataset.2d0a6a5b-6307-46e2-ad96-f16852648083"),
9 )    3 minutes ago · Uncommitted changes
10 def compute(features_df):
11
12     double_columns = [
13         "Temperature",
14         "Fuel_Price",
15         "MarkDown1",
16         "MarkDown2",
17         "MarkDown3",
18         "MarkDown4",
19         "MarkDown5",
20         "CPI",
21         "Unemployment"
22     ]
23
24     for column_name in double_columns:
25         features_df = features_df.withColumn(column_name, F.col(column_name).cast(DoubleType()))
26
27
28     # Convert Date to DateType
29     features_df = features_df.withColumn("Date", F.to_date(F.col("Date"), "yyyy-MM-dd"))    3 seconds
30
31     features_df = features_df.orderBy(F.col("Store").asc())
32
33     return features_df
34
```

```
converting_datatypes.py x
12     double_columns = [
13         "Unemployment"
14     ]
15
16     for column_name in double_columns:
17         features_df = features_df.withColumn(column_name, F.col(column_name).cast(DoubleType()))
18
19
20     # Convert Date to DateType
21     features_df = features_df.withColumn("Date", F.to_date(F.col("Date"), "yyyy-MM-dd"))    3 seconds
22
23     features_df = features_df.orderBy(F.col("Store").asc())
24
25     return features_df
26
```

Output –

Output

Datatypes ×

							Showing 8,190 rows	12 columns	🔍 Search columns...				
	Store 123 integer	✓	Date 📅 date	✓	Temperature 1.0 double	✓	Fuel_Price 1.0 double	✓	MarkDown1 1.0 double	✓	MarkDown2 1.0 double	✓	MarkDow 1.0 double
1		1	2010-02-05		42.31		2.572		null		null		
2		1	2010-02-12		38.51		2.548		null		null		
3		1	2010-02-19		39.93		2.514		null		null		
4		1	2010-02-26		46.63		2.561		null		null		
5		1	2010-03-05		46.50		2.625		null		null		
6		1	2010-03-12		57.79		2.667		null		null		
7		1	2010-03-19		54.58		2.720		null		null		
8		1	2010-03-26		51.45		2.732		null		null		
9		1	2010-04-02		62.27		2.719		null		null		
10		1	2010-04-09		65.86		2.770		null		null		
11		1	2010-04-16		66.32		2.808		null		null		
12		1	2010-04-23		64.84		2.795		null		null		
13		1	2010-04-30		67.41		2.780		null		null		
14		1	2010-05-07		72.55		2.835		null		null		
15		1	2010-05-14		74.78		2.854		null		null		
16		1	2010-05-21		76.44		2.826		null		null		
17		1	2010-05-28		80.44		2.759		null		null		

Calculate row count

Build –

★ Pinned The most important files in this project			0 items	
📁 Code Repository > Python > 📁 Output ☆ 📄 1			+ New ▾	
NAME ^	LAST UPDATED		TAGS	
📄 Converting_Datatypes ☆	Thu, Aug 27, 2026, 2:41:...			
📄 Handling_Missing_Values	Wed, Aug 26, 2026, 9:44:...			
📄 Renaming_Columns	Thu, Aug 27, 2026, 1:39:...			

Transformation 4 - Drop Unnecessary Columns and Add Composite Key:

Drop Markdown1–Markdown5 columns from Features if not needed for analysis. After dropping, create a new column store_type_key by combining Store and Type from Stores.

Code –

```
drop_unnecessary_columns.py x
1 from transforms.api import transform_df, Input, Output
2 from pyspark.sql import functions as F
3
4
5 @transform_df(
6     Output("/Accenture/Pratik_CapstoneProject/Code Repository/Python/Output/Drop_Unwanted_Columns"),
7     features_df = Input("Features", main.dataset.2d0a6a5b-6307-46e2-ad96-f16852648083),
8     stores_df = Input("Stores", main.dataset.bda7c97d-3b21-4978-a5b6-f6a5d4d3ec40")
9 )
10 def compute(features_df, stores_df):
11
12     # Drop markdown columns
13     features_df = features_df.drop(
14         "Markdown1",
15         "Markdown2",
16         "Markdown3",
17         "Markdown4",
18         "Markdown5"
19     )
20
21     # Select required Store attributes
22     stores_df = stores_df.select("Store", "Type")
23
```

```
drop_unnecessary_columns.py x
19 )
20
21     # Select required Store attributes
22     stores_df = stores_df.select("Store", "Type")
23
24     # Features with Stores 3 seconds ago - Uncommitted changes
25     result_df = features_df.join(stores_df, on="Store", how="inner")
26
27     # Create store_type_key
28     result_df = result_df.withColumn(
29         "store_type_key",
30         F.concat_ws("_", F.col("Store").cast("string"), F.col("Type"))
31     )
32
33
34     return result_df
```

Output –

Output

Drop_Unwanted_Columns

Showing 8,190 rows

9 columns

Search columns...

	Store integer	Date date	Temperature double	Fuel_Price double	CPI string	Unemployment string	IsHoliday boolean
1	17	2011-10-14	47.69	3.567	129.7706452	6.617	false
2	17	2011-10-21	47.27	3.579	129.7821613	6.617	false
3	17	2011-10-28	41.89	3.567	129.7936774	6.617	false
4	17	2011-11-04	37.52	3.538	129.8051935	6.617	false
5	17	2011-11-11	27.61	3.513	129.8167097	6.617	false
6	17	2011-11-18	32.93	3.489	129.8268333	6.617	false
7	17	2011-11-25	32.81	3.445	129.8364	6.617	true
8	17	2011-12-02	26.47	3.389	129.8459667	6.617	false
9	17	2011-12-09	17.56	3.341	129.8555333	6.617	false
10	17	2011-12-16	21.07	3.282	129.8980645	6.617	false
11	17	2011-12-23	18.76	3.186	129.9845484	6.617	false
12	17	2011-12-30	26.73	3.119	130.0710323	6.617	true
13	17	2012-01-06	27.64	3.080	130.1575161	6.403	false
14	17	2012-01-13	20.28	3.056	130.244	6.403	false
15	17	2012-01-20	27.26	3.047	130.2792258	6.403	false
16	17	2012-01-27	29.99	3.058	130.3144516	6.403	false
17	17	2012-02-03	26.26	3.077	130.2406774	6.403	false

Calculate row count

Output

Drop_Unwanted_Columns

Showing 8,190 rows

9 columns

	Fuel_Price 1.0 double	CPI 99 string	Unemployment 99 string	IsHoliday boolean	Type 99 string	store_type_key 99 string
1	47.69	3.567	129.7706452	6.617	false	B
2	47.27	3.579	129.7821613	6.617	false	B
3	41.89	3.567	129.7936774	6.617	false	B
4	37.52	3.538	129.8051935	6.617	false	B
5	27.61	3.513	129.8167097	6.617	false	B
6	32.93	3.489	129.8268333	6.617	false	B
7	32.81	3.445	129.8364	6.617	true	B
8	26.47	3.389	129.8459667	6.617	false	B
9	17.56	3.341	129.8555333	6.617	false	B
10	21.07	3.282	129.8980645	6.617	false	B
11	18.76	3.186	129.9845484	6.617	false	B
12	26.73	3.119	130.0710323	6.617	true	B
13	27.64	3.080	130.1575161	6.403	false	B
14	20.28	3.056	130.244	6.403	false	B
15	27.26	3.047	130.2792258	6.403	false	B
16	29.99	3.058	130.3144516	6.403	false	B
17	26.26	3.077	130.2406774	6.403	false	B

Calculate row count

Build –

Pinned

The most important files in this project

0 items

>

Code Repository

>

Python

>

Output

1

+ New

NAME	LAST UPDATED	TAGS
Converting_Datatypes	Thu, Aug 27, 2026, 2:41:...	
Drop_Unwanted_Columns	Thu, Aug 27, 2026, 12:11:...	
Handling_Missing_Values	Wed, Aug 26, 2026, 9:44:...	
Renaming_Columns	Thu, Aug 27, 2026, 1:39:...	

Transformation 5 - Create Derived Column and Unique Department Key:

After joining Sales and Stores, create a new column `sales_per_sqft` by dividing `Weekly_Sales` by `Size`. Then, add a column `store_dept_key` by concatenating `Store` and `Dept` to uniquely identify each department-level record.

Code –

```
derived_column_unique_dept_key.py ×
1  from transforms.api import transform_df, Input, Output
2  from pyspark.sql import functions as F
3
4
5  @transform_df(
6      Output("/Accenture/Pratik_CapstoneProject/Code Repository/Python/Output/Joining_Sales_Store"),
7      sales_df = Input("Sales", main.dataset.0bffffb95-2df3-4bcf-9dea-e3b48b6cb7f9),
8      stores_df = Input("Stores", main.dataset.bda7c97d-3b21-4978-a5b6-f6a5d4d3ec40")
9  )
10 def compute(sales_df, stores_df):
11
12     # Join Sales and Stores using Store
13     joined_df = sales_df.join(
14         stores_df,
15         on="Store",
16         how="inner"
17     ) 1 minute ago · Uncommitted changes
18
19     # Create sales_per_sqft
20     joined_df = joined_df.withColumn(
21         "sales_per_sqft",
22         F.when(
23             F.col("Size") != 0,
24             F.col("Weekly_Sales") / F.col("Size")
25
26
27
28     # Create store_dept_key
29     joined_df = joined_df.withColumn(
30         "store_dept_key",
31         F.concat_ws(
32             "_", 3 minutes ago · Uncommitted changes
33             F.col("Store").cast("string"),
34             F.col("Dept").cast("string")
35         )
36     )
37
38     return joined_df
```

Output —

Output Joining_Sales_Store ×								
						Showing 10,000 rows	9 columns	🔍 Search columns...
	Store 123 integer	Dept 123 integer	Date date	Weekly_Sales 1.0 double	IsHoliday boolean	Type string	Size 123 integer	
1		1	90	2011-03-25	74628.50	false	A	
2		1	90	2011-04-01	74995.00	false	A	
3		1	90	2011-04-08	83707.82	false	A	
4		1	90	2011-04-15	81817.98	false	A	
5		1	90	2011-04-22	80709.38	false	A	
6		1	90	2011-04-29	75655.35	false	A	
7		1	90	2011-05-06	85337.89	false	A	
8		1	90	2011-05-13	81465.80	false	A	
9		1	90	2011-05-20	74840.01	false	A	
10		1	90	2011-05-27	76414.57	false	A	
11		1	90	2011-06-03	83651.06	false	A	
12		1	90	2011-06-10	85613.47	false	A	
13		1	90	2011-06-17	84721.24	false	A	
14		1	90	2011-06-24	78145.72	false	A	
15		1	90	2011-07-01	81547.54	false	A	
16		1	90	2011-07-08	81734.19	false	A	

Output Joining_Sales_Store ×								
						Showing 10,000 rows	9 columns	🔍 Search columns...
	Weekly_Sales 1.0 double	IsHoliday boolean	Type string	Size 123 integer	sales_per_sqft 1.0 double	store_dept_key string		
1	11-03-25	74628.50	false	A	151315	0.4931996166936523000	1_90	
2	11-04-01	74995.00	false	A	151315	0.4956217162872154000	1_90	
3	11-04-08	83707.82	false	A	151315	0.5532023923603080000	1_90	
4	11-04-15	81817.98	false	A	151315	0.5407129498066946000	1_90	
5	11-04-22	80709.38	false	A	151315	0.5333865115817996000	1_90	
6	11-04-29	75655.35	false	A	151315	0.4999857912302151400	1_90	
7	11-05-06	85337.89	false	A	151315	0.494597429203978400000	1_90	
8	11-05-13	81465.80	false	A	151315	0.494597429203978400000	1_90	
9	11-05-20	74840.01	false	A	151315	0.4945974292039784000	1_90	
10	11-05-27	76414.57	false	A	151315	0.5050032713214156000	1_90	
11	11-06-03	83651.06	false	A	151315	0.5528272808379869000	1_90	
12	11-06-10	85613.47	false	A	151315	0.5657963189373162000	1_90	
13	11-06-17	84721.24	false	A	151315	0.5598998116511913000	1_90	
14	11-06-24	78145.72	false	A	151315	0.5164439744903017000	1_90	
15	11-07-01	81547.54	false	A	151315	0.5389256848296600000	1_90	

Build -

★ Pinned The most important files in this project			0 items
📁 > Code Repository > Python > 📁 Output ☆ 📄 1			+ New ▾
NAME ^	LAST UPDATED	TAGS	
📄 Converting_Datatypes ☆	Thu, Aug 27, 2026, 2:41:...		
📄 Drop_Unwanted_Columns	Thu, Aug 27, 2026, 12:11:...		
📄 Handling_Missing_Values	Wed, Aug 26, 2026, 9:44:...		
📄 Joining_Sales_Store ☆	Thu, Aug 27, 2026, 12:44:...		
📄 Renaming_Columns	Thu, Aug 27, 2026, 1:39:...		

❖ Data Integration using Pipelines

The Pipeline Builder in Foundry allows users to design and execute data transformation workflows without writing code.

Steps:

1. Import the Stores, Sales, and Features datasets.
2. Handle missing values and validate data types.
3. Create primary key columns such as Feature_ID and Sales_ID.
4. Standardize column names such as Store to Store_ID.
5. Join Features with Stores and Sales with Stores.
6. Create the final cleansed output datasets.
7. Deploy the pipeline successfully.

1. Data Cleansing Pipeline ➤ Oversee Missing Values: Replace NA or null values in Markdown1–5, CPI and Unemployment columns with 0 in Feature dataset.

The screenshot displays the Foundry Pipeline Builder interface for a dataset named 'Features' with 12 columns. The pipeline consists of 15 steps, each with a name, a target column, and a set of action icons (Edit, Preview, Save, Copy, Delete).

Step	Operation	Target Column	Actions
1	Regex replace	Markdown1	Edit, Preview, Save, Copy, Delete
2	Regex replace	Markdown2	Edit, Preview, Save, Copy, Delete
3	Regex replace	Markdown3	Edit, Preview, Save, Copy, Delete
4	Regex replace	Markdown4	Edit, Preview, Save, Copy, Delete
5	Regex replace	Markdown5	Edit, Preview, Save, Copy, Delete
6	Regex replace	CPI	Edit, Preview, Save, Copy, Delete
7	Regex replace	Unemployment	Edit, Preview, Save, Copy, Delete
8	Cast to Double	Temperature	Edit, Preview, Save, Copy, Delete
9	Cast to Double	Markdown1	Edit, Preview, Save, Copy, Delete
10	Cast to Double	Markdown2	Edit, Preview, Save, Copy, Delete
11	Cast to Double	Markdown3	Edit, Preview, Save, Copy, Delete
12	Cast to Double	Markdown4	Edit, Preview, Save, Copy, Delete
13	Cast to Double	Markdown5	Edit, Preview, Save, Copy, Delete

Cast to Double → Markdown5

Edit Preview

Cast to Double → Unemployment

Edit Preview

Cast to Double → CPI

Edit Preview

Cast to Double → Fuel_Price

Edit Preview

"1" → partition_key

Edit Preview

WINDOW

Edit Preview

Window: Sorted

Expressions: 1 value

Drop columns: partition_key

Edit Preview

Cast to String → Feature_ID

Edit Preview

Cast to String → Store

Edit Preview

Rename: 1 column

Edit Preview

Search transforms and columns...

Generate

No inputs sampled

	Store_ID string	Feature_ID string	Fuel_Price 1.0 double	CPI 1.0 double	Unemployment 1.0 double	Markdown5 1.0 double
1	1	1	3.620		0	1245.20
2	2	2	3.620		0	2416.20
3	3	3	3.620		0	1152.08
4	4	4	3.580		0	1433.77
5	5	5	3.620		0	948.51
6	6	6	3.620		0	2648.37
7	7	7	3.582		0	877.99
8	8	8	3.620		0	2343.11
9	9	9	3.620		0	1265.60
10	10	10	3.865		0	1649.64
11	11	11	3.620		0	3264.52
12	12	12	3.865		0	2132.08
13	13	13	3.669		0	2147.06
14	14	14	3.804		0	1972.09
15	15	15	3.951		0	1192.82
16	16	16	3.582		0	1795.21
17	17	17	3.669		0	921.75

No inputs sampled

	Markdown4 0 double	Markdown3 1.0 double	Markdown2 1.0 double	Markdown1 1.0 double	Temperature 1.0 double	Date date	IsHoliday boolean
1	38.63	1.00	673.19	332.17	81.54	2013-07-26	false
2	493.60	91.00	609.55	1966.46	83.17	2013-07-26	false
3	88.91	0.00	256.64	586.26	85.53	2013-07-26	false
4	314.64	9.60	687.89	1577.60	78.96	2013-07-26	false
5	0.00	0.09	35.39	131.42	84.16	2013-07-26	false
6	70.40	16.05	798.02	543.69	84.42	2013-07-26	false
7	36.65	0.00	54.56	180.87	56.69	2013-07-26	false
8	73.42	55.38	740.04	675.56	78.70	2013-07-26	false
9	2.50	0.00	258.14	52.43	82.37	2013-07-26	false
10	198.93	165.50	936.36	1221.15	83.32	2013-07-26	false
11	1084.84	106.70	717.20	2387.21	85.64	2013-07-26	false
12	284.49	6.00	232.99	1008.91	90.27	2013-07-26	false
13	140.32	93.40	1377.41	346.31	83.62	2013-07-26	false
14	185.27	13.90	731.33	779.98	76.06	2013-07-26	false
15	130.40	0.00	1754.52	312.11	68.77	2013-07-26	false
16	272.07	0.00	102.26	375.42	67.29	2013-07-26	false
17	73.46	11.00	473.39	64.25	71.41	2013-07-26	false

2. Validate Data Types: Ensure numeric fields such as Weekly_Sales, Temperature, Fuel_Price, MarkDown 1-5, CPI, and Unemployment from Feature and Sales are stored as Double.

Sales_Cleansed

Expand allAppliedClose

Sales5 columns

Cast to Double → Weekly_Sales

EditPreview

"1" → partition_key

EditPreview

WINDOW

EditPreview

Window: Sorted

Expressions: 1 value

Cast to String → Sales_ID

EditPreview

Drop columns: partition_key

EditPreview

Cast to String → Store

EditPreview

Rename: 1 column

EditPreview

Search transforms and columns...

Generate

Previewing 500 rows

6 columns

Search columns...

No inputs sampled

	Store_ID string	Sales_ID string	Weekly_Sales 1.0 double	Dept 123 integer
1	1	1	27390.81	1
2	1	2	43134.88	2
3	1	3	9350.90	3
4	1	4	36292.60	4
5	1	5	25846.94	5
6	1	6	4032.50	6
7	1	7	21449.85	7
8	1	8	39255.35	8
9	1	9	41040.84	9
10	1	10	26285.27	10
11	1	11	22399.83	11
12	1	12	8658.41	12

Previewing 500 rows

6 columns

Search columns...

No inputs sampled

	Weekly_Sales 1.0 double	Dept 123 integer	Date date	IsHoliday boolean
1	27390.81	1	2012-10-26	false
2	43134.88	2	2012-10-26	false
3	9350.90	3	2012-10-26	false
4	36292.60	4	2012-10-26	false
5	25846.94	5	2012-10-26	false
6	4032.50	6	2012-10-26	false
7	21449.85	7	2012-10-26	false
8	39255.35	8	2012-10-26	false
9	41040.84	9	2012-10-26	false
10	26285.27	10	2012-10-26	false
11	22399.83	11	2012-10-26	false

3. Create Key Attributes: Create Primary Key column in Feature and Sales dataset namely Feature_ID and Sales_ID in String datatype
4. Column Rename: Rename the column named Store to Store_ID in all three datasets and make it String

Stores 3 columns

Rename: 1 column
Edit Preview

Cast to String → Store_ID
Edit Preview

Search transforms and columns...
Generate

Previewing 45 rows

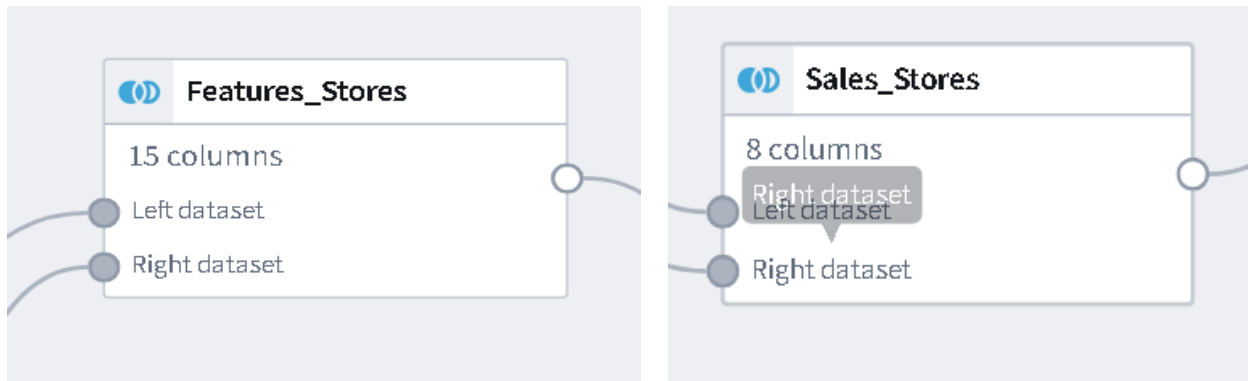
3 columns

Search columns...

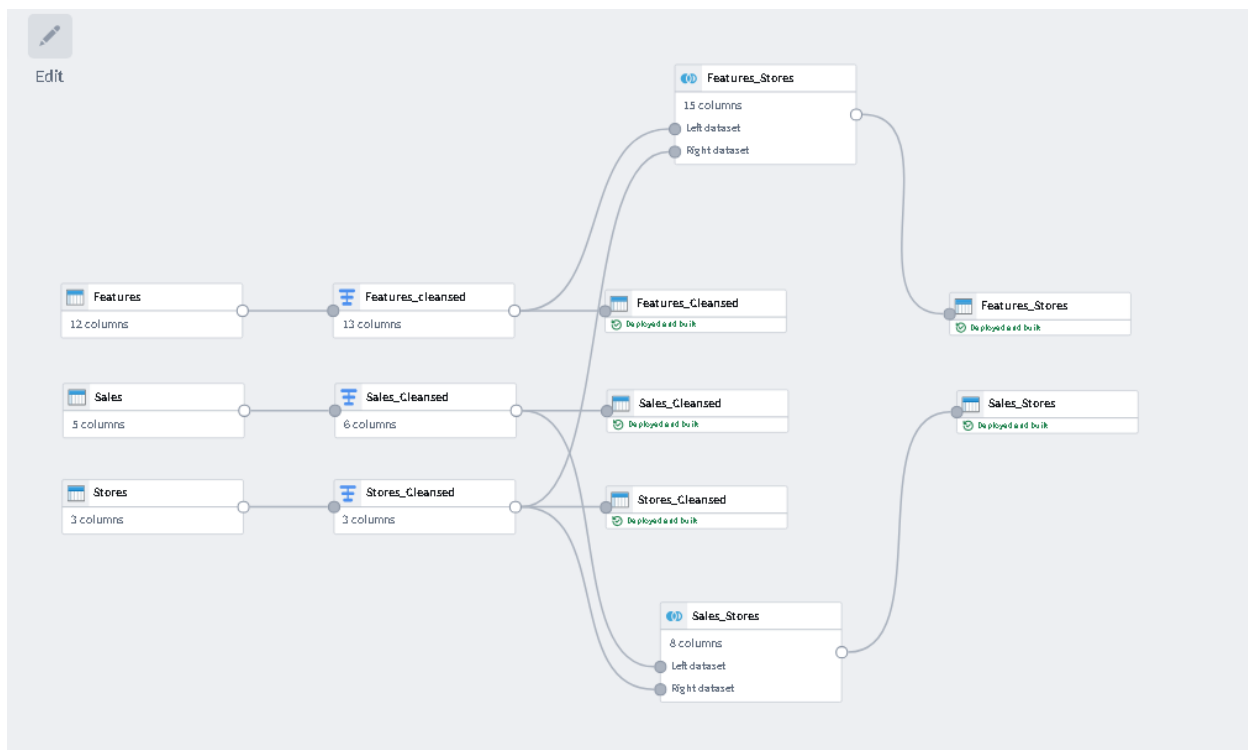
No inputs sampled

	Store_ID string	Type string	Size integer
1	1	A	1513
2	2	A	2023
3	3	B	373
4	4	A	2058
5	5	B	348
6	6	A	2025
7	7	B	707
8	8	A	1550
9	9	B	1258
10	10	B	1265
11	11	A	2074
12	12	B	1122

5. Combine Data: Combine the above transformed databased on key attributes and create two more datasets namely Features_Stores and Sales_Stores.



Final Pipeline –



Pipeline Builder > Output Datasets ☆ 1			Actions
NAME ^	LAST UPDATED	TAGS	
 Features_Cleansed	Thu, Aug 27, 2026, 4:19:44 PM		
 Features_Stores	Thu, Aug 27, 2026, 4:19:46 PM		
 Sales_Cleansed	Thu, Aug 27, 2026, 4:19:41 PM		
 Sales_Stores	Thu, Aug 27, 2026, 4:19:50 PM		
 Stores_Cleansed	Thu, Aug 27, 2026, 4:19:38 PM		

❖ Create and explore Objects using Ontology Manager

Component Utilization: Ontology Manager translated tabular data into a digital twin of the retail environment, allowing applications like Workshop to query objects and relationships rather than raw tables.

Steps:

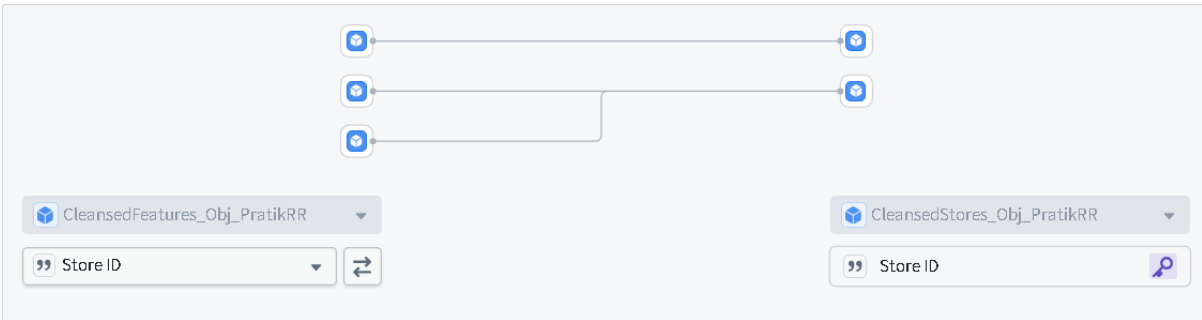
1. Create Object Types for Cleansed Stores, Sales, and Features.
 2. Map the corresponding datasets to the Object Types.
 3. Configure the Primary Key and Title Key.
 4. Create relationships between the objects.
 5. Establish one-to-many relationships from Stores to Sales and Features.
 6. Verify the object data and relationships.
 7. Use Ontology objects for Quiver analysis and Workshop applications.
1. Object Creation: Create three core business objects namely CleansedFeatures_Obj_, CleansedSales_Obj_, CleansedStores_Obj_ using the Feature_cleansed, Sales_Cleansed and Store_Cleansed datasets.

Pinned The most important files in this project 0 items		
Ontology Objects ☆ 📅 1 Actions + New ▼		
NAME ^	LAST UPDATED	TAGS
CleansedFeatures_Obj_PratikRR	Thu, Aug 27, 2026, 7:03:...	
CleansedSales_Obj_PratikRR	Thu, Aug 27, 2026, 6:59:...	
CleansedStores_Obj_PratikRR	Thu, Aug 27, 2026, 7:05:...	

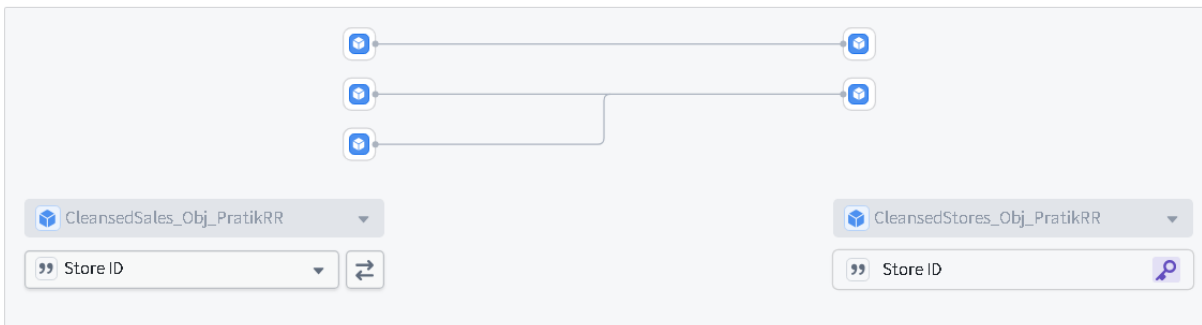
2. Link Creation: Established one-to-many relationships between:
- Stores → Sales (via Store_ID)
 - Stores → Features (via Store_ID)

Pinned The most important files in this project 0 items		
Ontology Links ☆ 📅 1 Actions + New ▼		
NAME ^	LAST UPDATED	TAGS
CleansedStores_Obj_PratikRR - Cleanse...	Thu, Aug 27, 2026, 7:03:...	
CleansedStores_Obj_PratikRR - Cleanse...	Thu, Aug 27, 2026, 6:59:...	

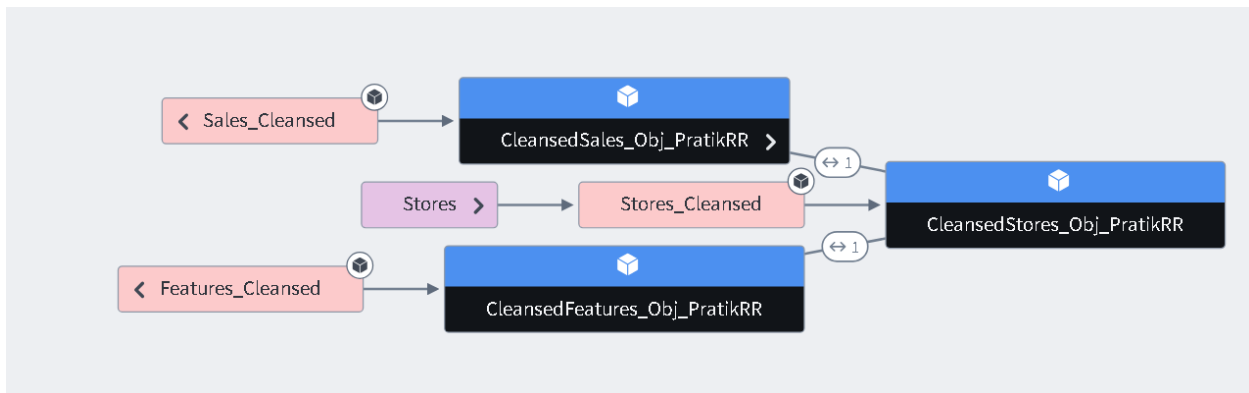
Features - Stores



Sales - Stores



Data Lineage –



❖ Data Analysis using Quiver –

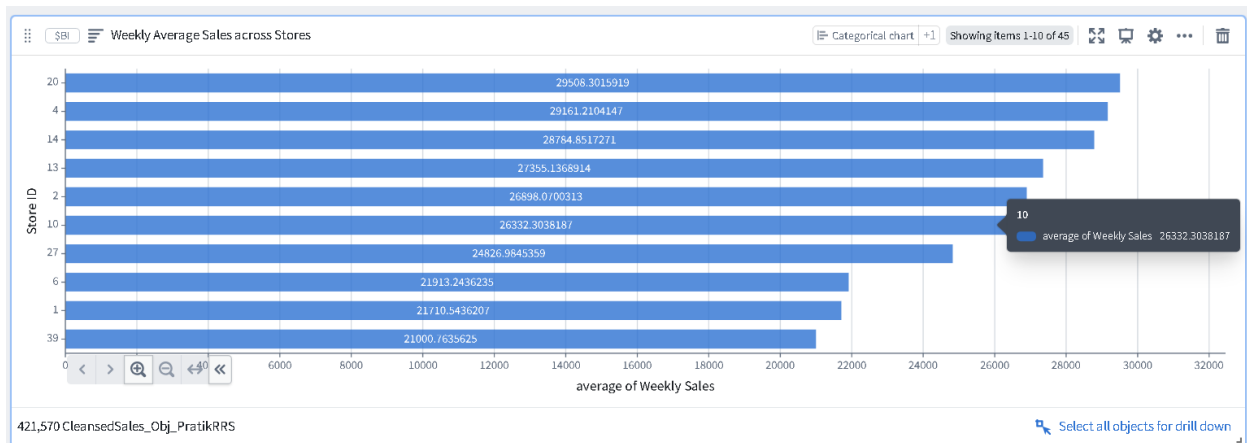
Component Utilization: Quiver was used to perform object-based analysis and visual aggregations, leveraging the ontology links to find correlations between store properties and sales metrics.

Steps:

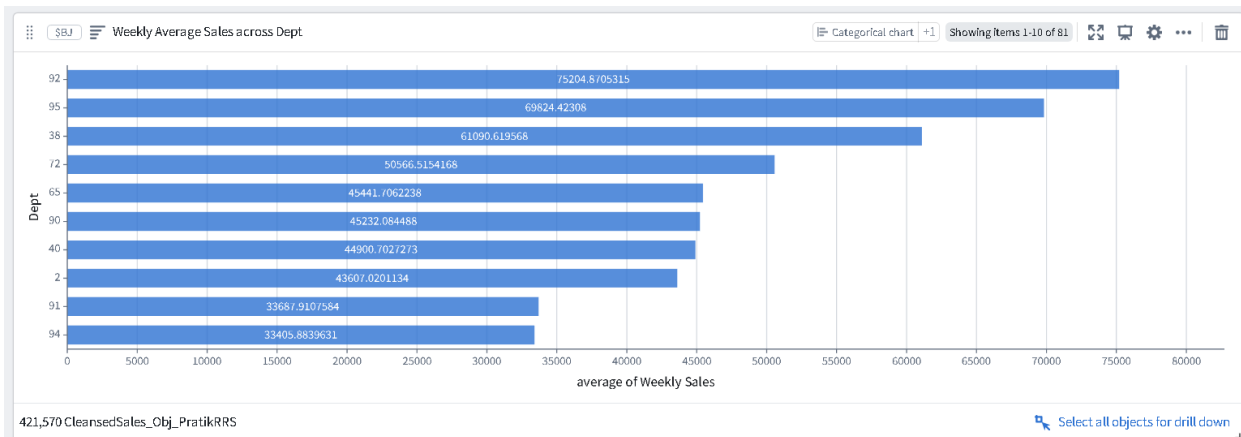
1. Import the CleansedSales and CleansedStores objects.
2. Analyze weekly sales across stores, departments, and dates.
3. Apply filters such as Store ID, Date Range, and IsHoliday.
4. Perform aggregations to identify top-performing stores.
5. Analyze relationships such as Store Type vs. Fuel Price.
6. Analyze holiday sales by department.
7. Analyze the impact of temperature on store performance.
8. Combine the analysis cards into a Quiver Canvas.

1. Analyze Sales Performance: Compare Weekly_Sales across stores, departments, and time periods to identify top performing stores or departments.

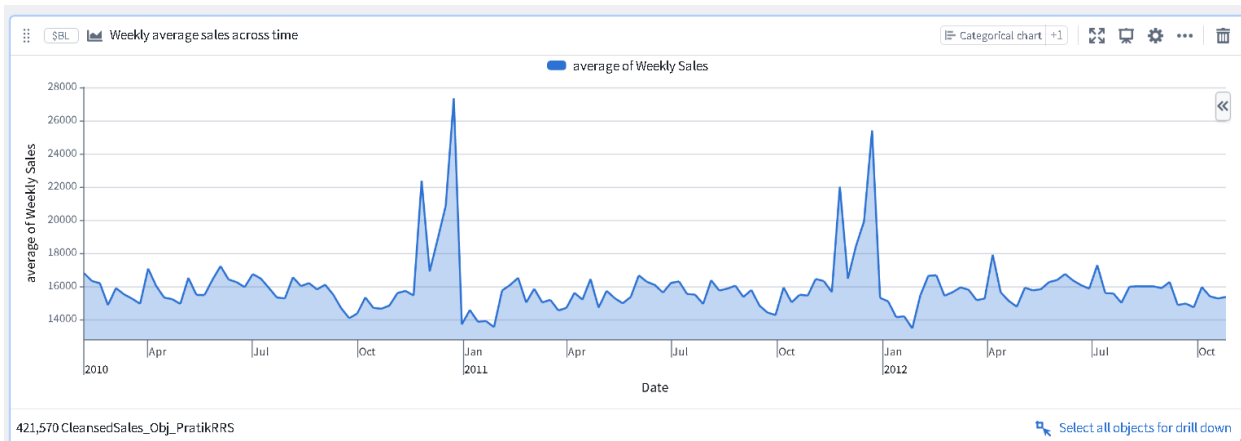
- Weekly sales across stores –



Weekly sales by departments –

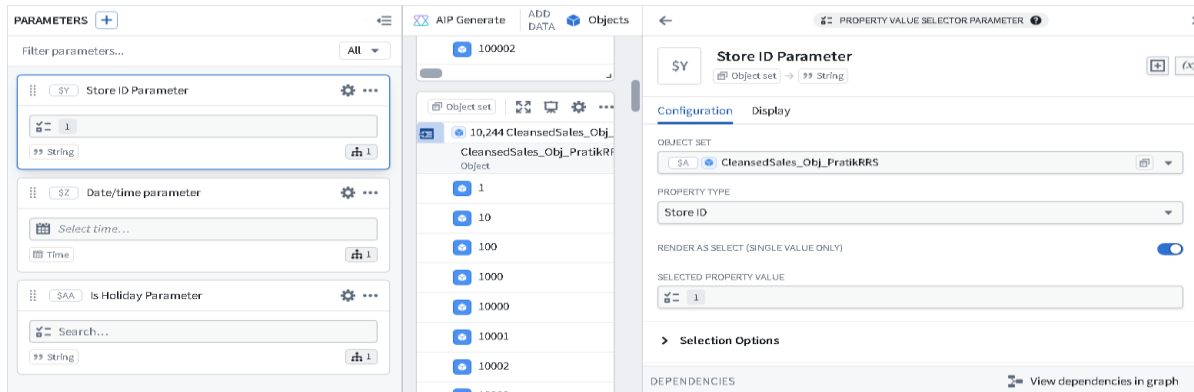


- Weekly sales by Date/time –

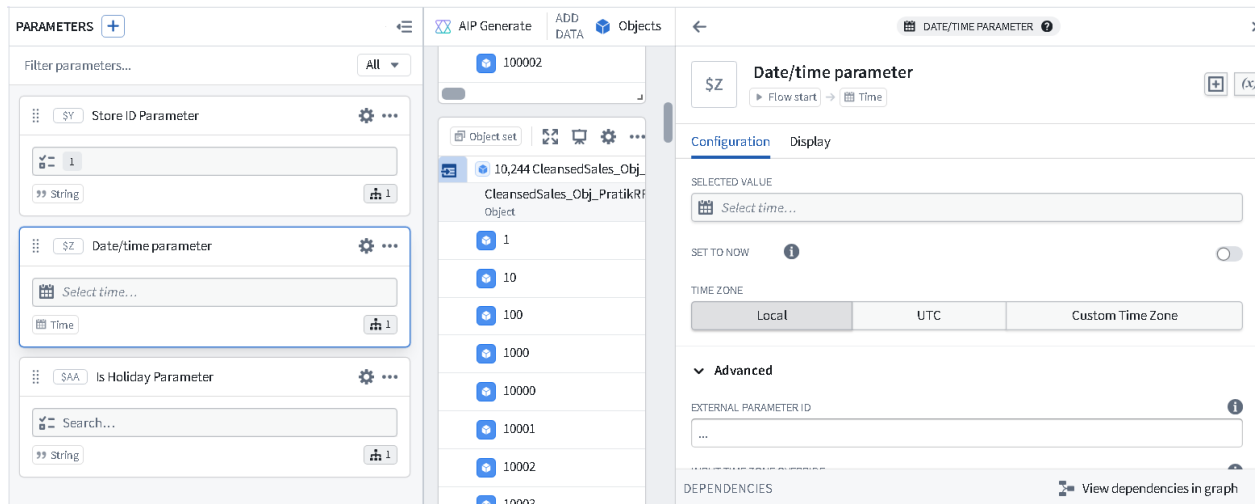


2. Apply Filters and Parameters: Add parameters such as Store ID, Date Range, or IsHoliday to allow dynamic exploration of dashboards

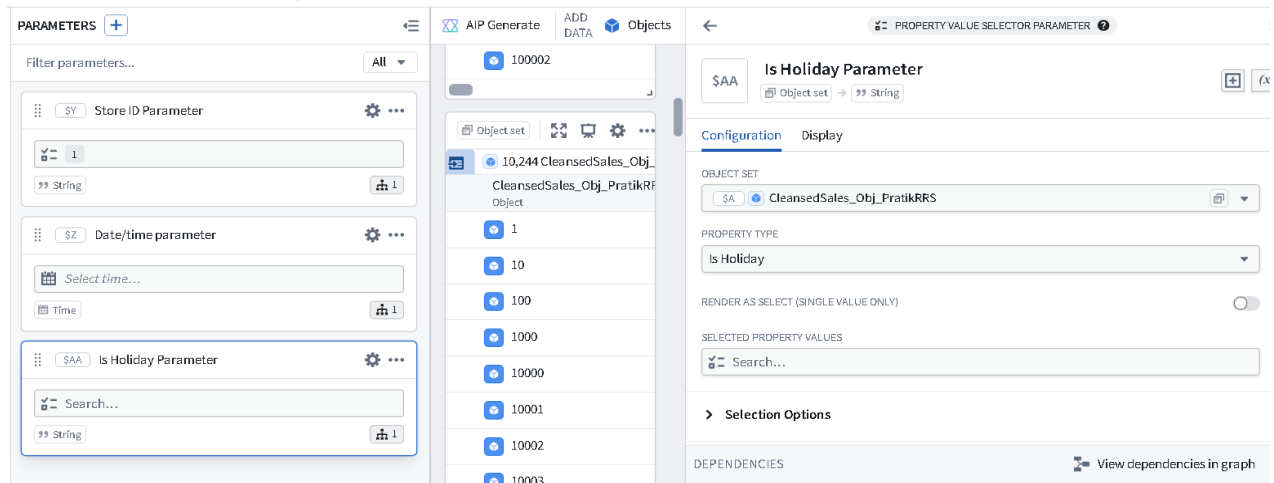
Store ID -



Date/Time -



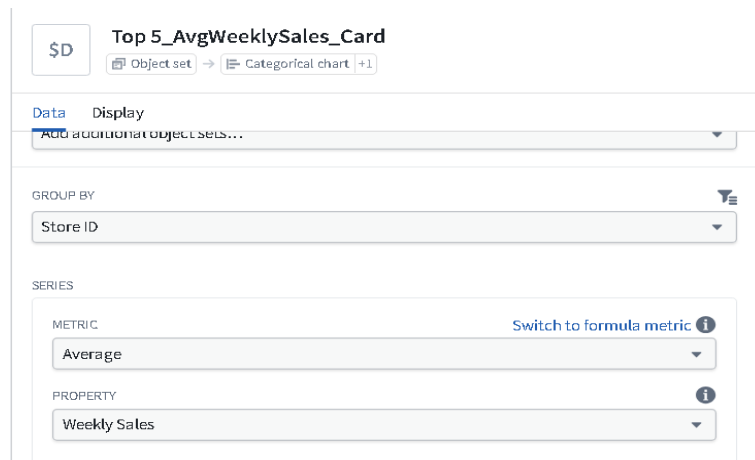
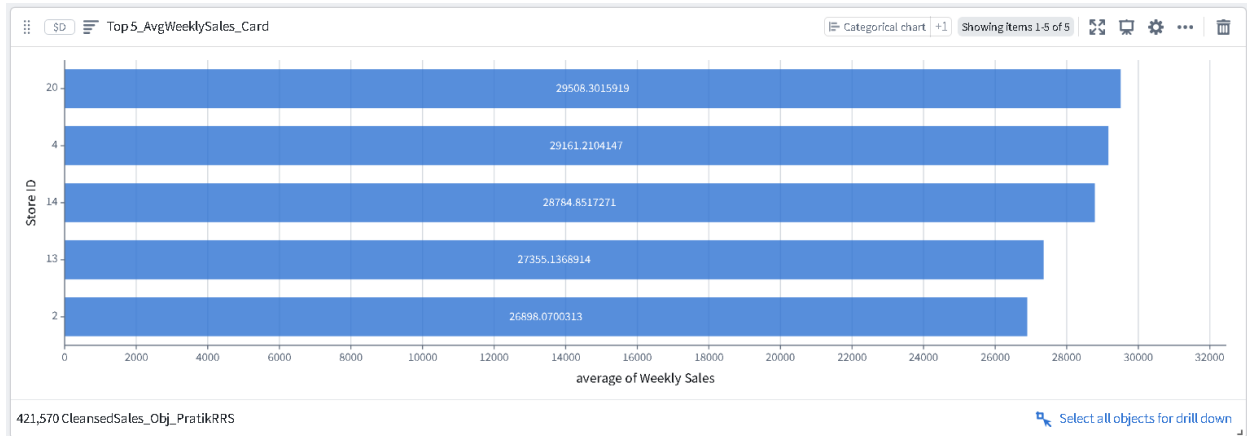
IsHoliday -



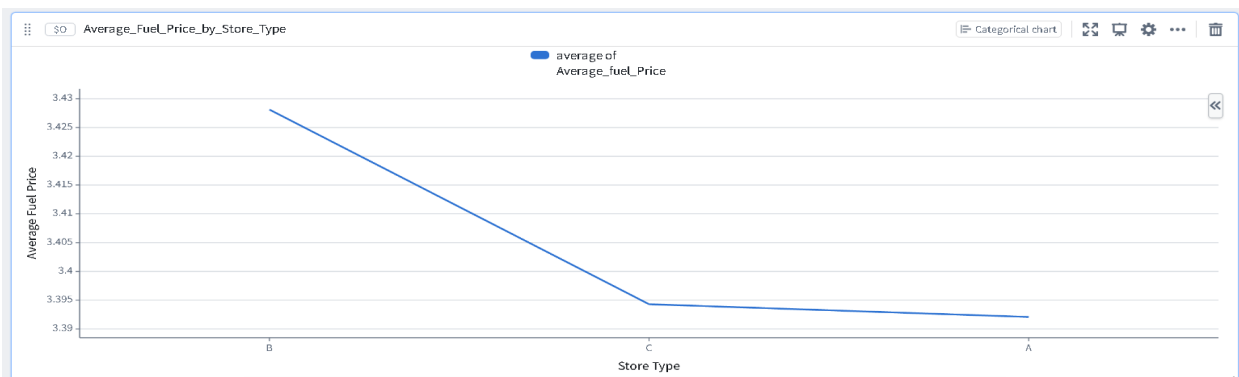
Perform aggregated queries & Insight Extraction:

Execute analysis on linked objects to derive insights like:

1. Top 5 stores with highest weekly average sales.



2. Store Type vs. Average Fuel_Price correlation. –



\$0

Average_Fuel_Price_by_Store_Type

Transform table → Categorical chart

Data

Display

INPUT TABLE

\$L

Transform table

GROUP BY

linked_type_group_name

SERIES

METRIC

Average

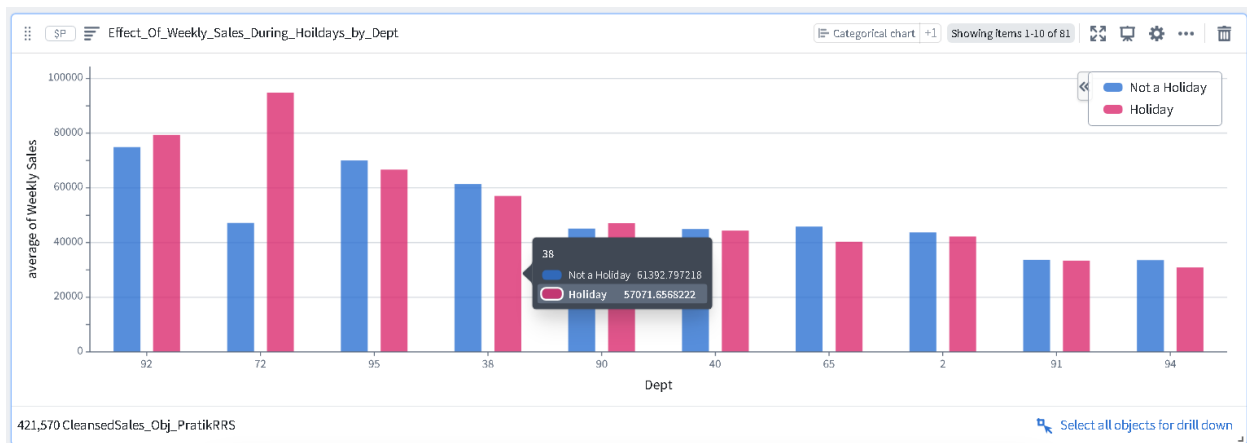
Sv

PROPERTY

123

Average_fuel_Price

3. Effect of Weekly_Sales during holidays on each dept.



\$P

Effect_Of_Weekly_Sales_During_Holidays_by_Dept

Object set → Categorical chart +1

Data

Display

Dept

Exact Value

SERIES

METRIC

Average

Switch to formula metric

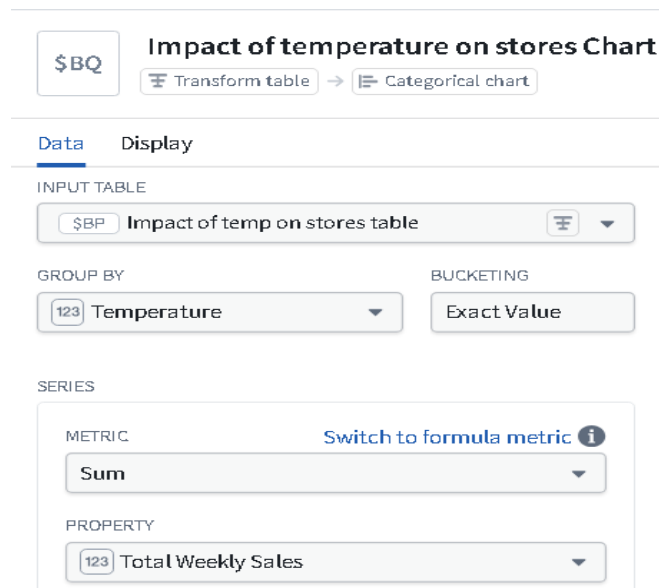
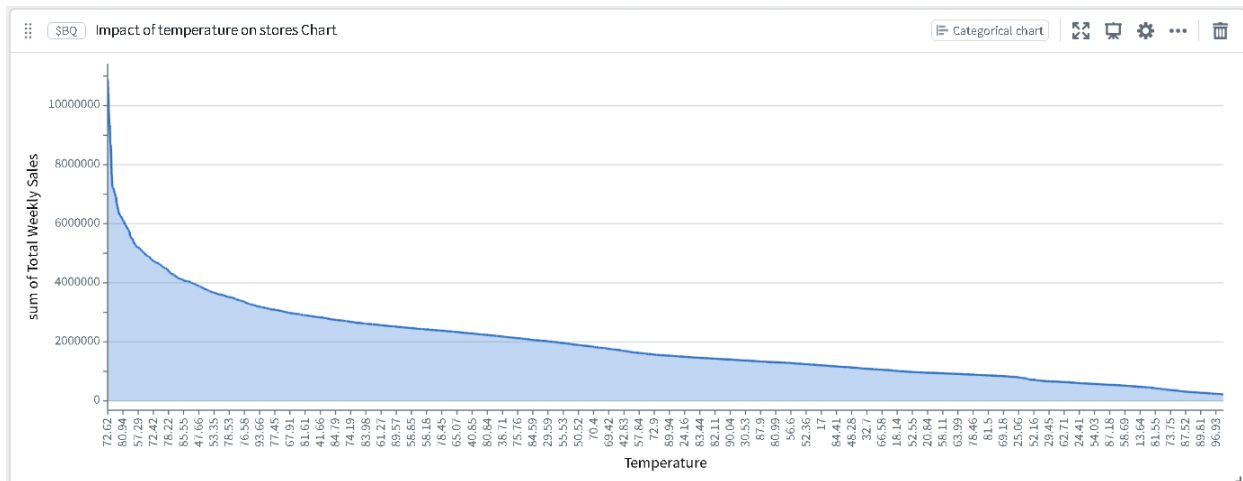
PROPERTY

Weekly Sales

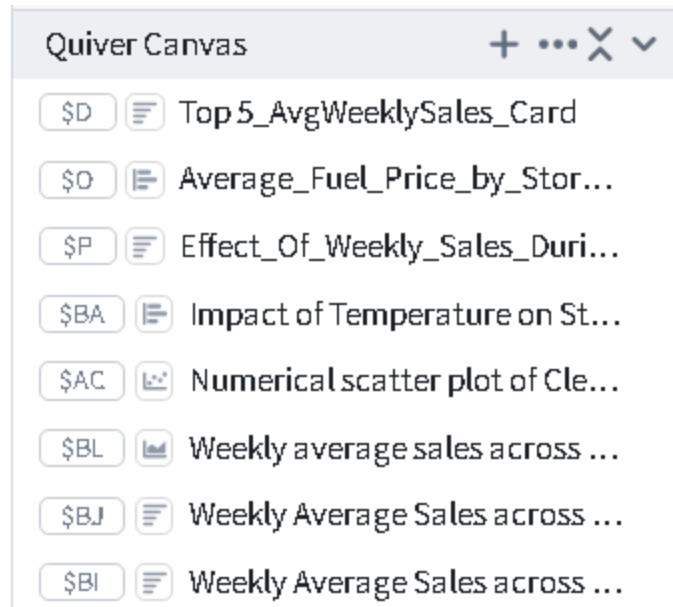
SEGMENT BY

Is Holiday

4. Impact of Temperature on Store Performance



3. Quiver Canvas: Combine the above analysis cards into a single Quiver Canvas to visualize overall patterns and correlations. Perform resizing, provide title wherever required.



❖ Data Visualization using Contour –

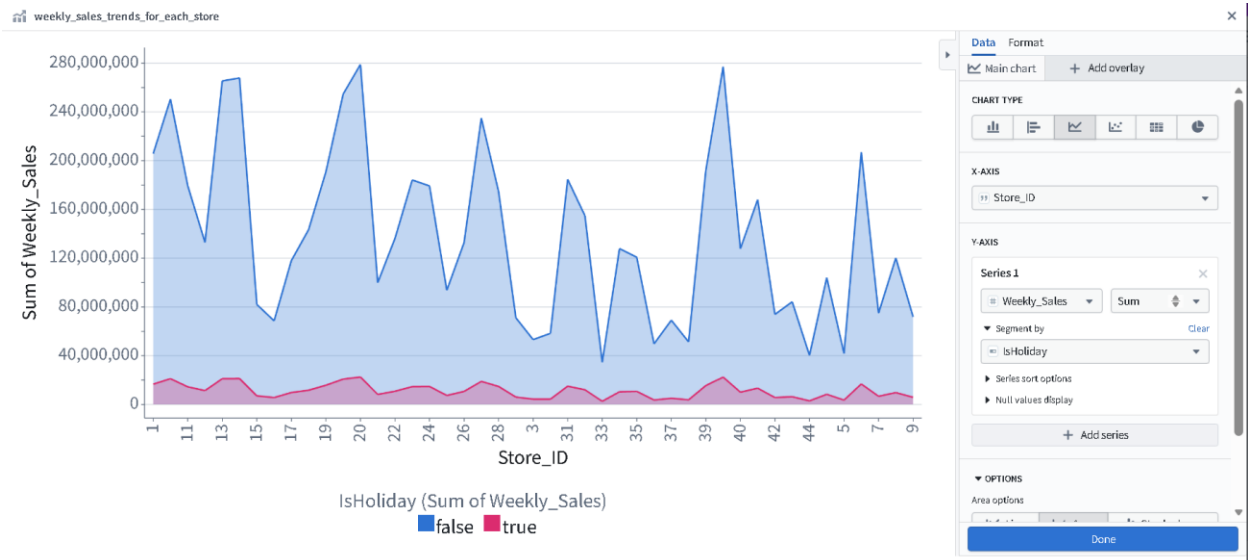
Component Utilization: Contour was used for top-down analytical pathways, aggregating millions of rows into summarized interactive dashboards to detect temporal trends.

Steps:

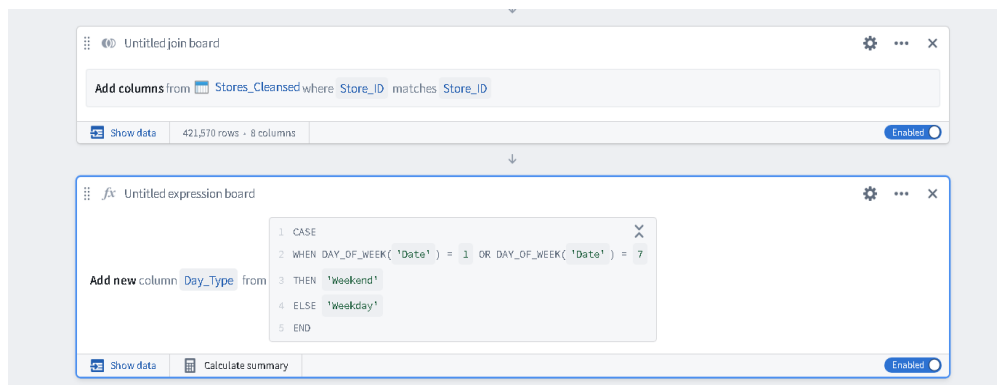
1. Connect the required Ontology objects/data.
2. Create a sales trend line chart over time.
3. Compare store performance using bar charts or heatmaps.
4. Create summary cards for total and average sales.
5. Visualize the relationship between Temperature and Weekly Sales.
6. Identify the top-performing stores.
7. Add interactive filters for Store ID, Date Range, and IsHoliday.
8. Arrange the visualizations into an interactive dashboard.

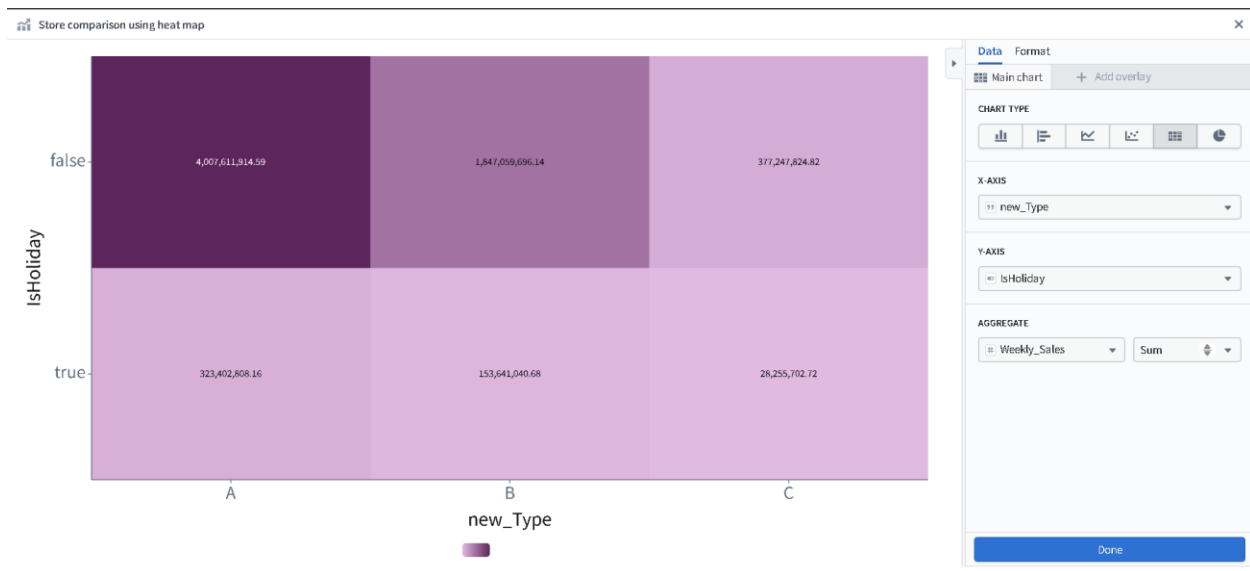
Data Visualization

- Sales Trend Analysis: Create a line chart to display Weekly_Sales trends over time for each store, highlighting holiday weeks using distinct colors or markers



- Store Comparison: Design a bar or heatmap chart to compare total weekly sales across different store types during weekends and weekdays using attributes from Stores Object and Sales Object.

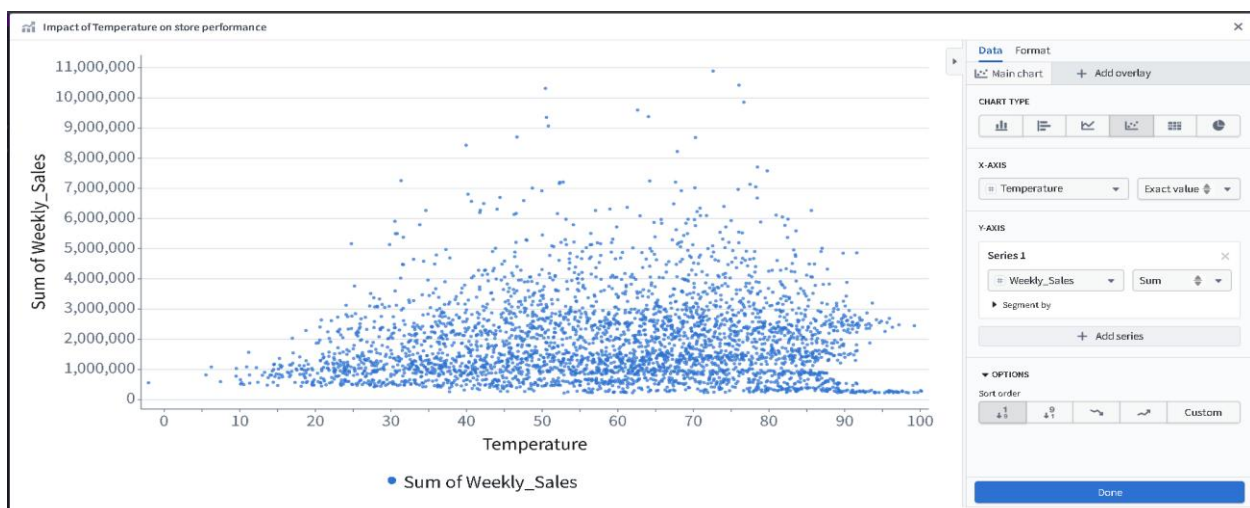




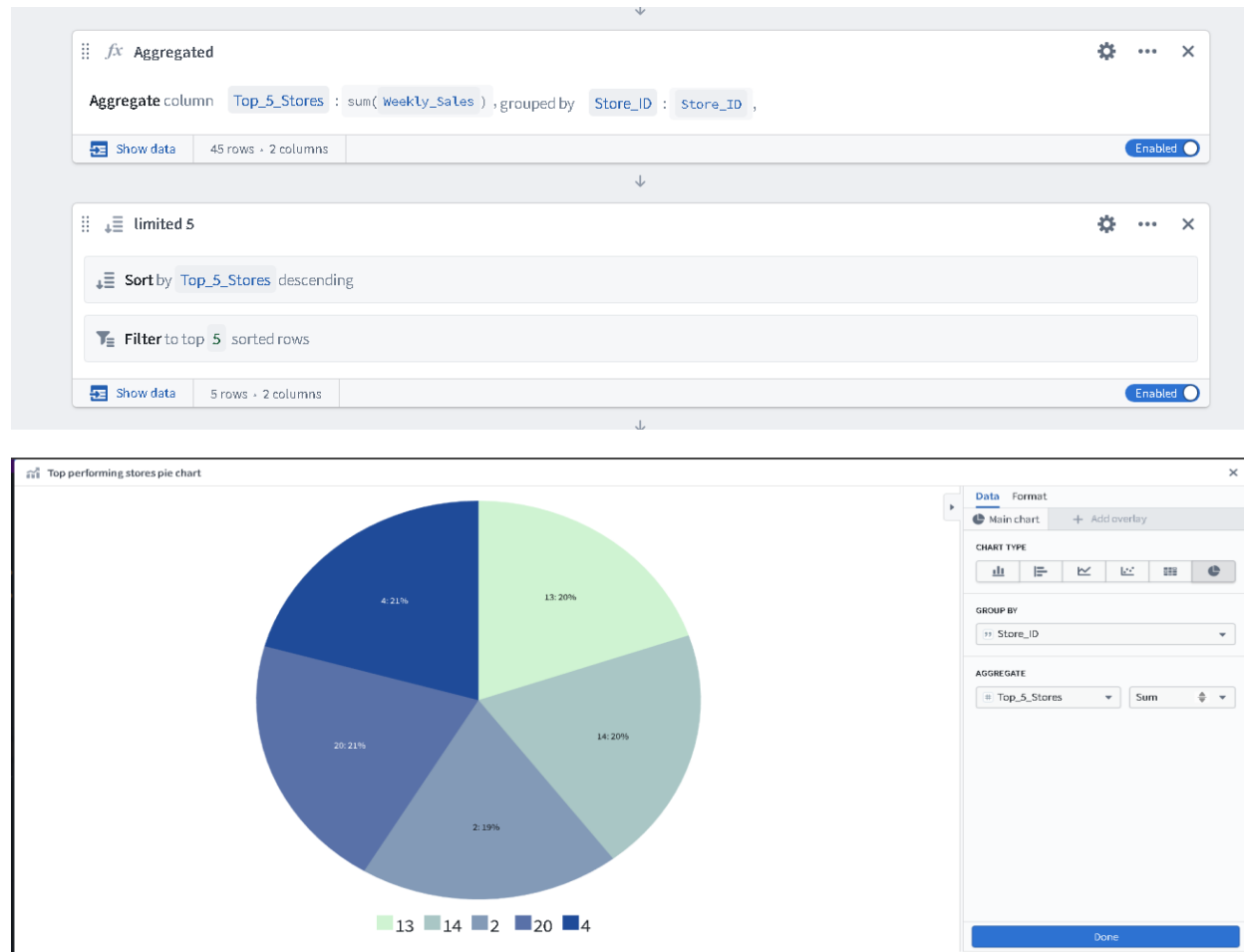
➤ Summary card: Display the total weekly sales using Calculation board



➤ Impact of Temperature on Store Performance: Display the relationship between Temperature and Weekly_Sales over week using Scatter plot or Bubble chart



- Top Performing Stores: Showcase the Top 5 stores with the highest total sales using pie chart



- Interactive Filters: Add dynamic parameters such as Store_ID, Date_Range, and IsHoliday to allow users to explore and filter visualizations interactively

Parameters

Use parameters in filters and expressions to quickly manipulate data by typing "\$".
Filtering will be ignored when no value is set for a parameter.

[+ Create parameter](#)

\$Store

\$Date_Range

\$Holiday

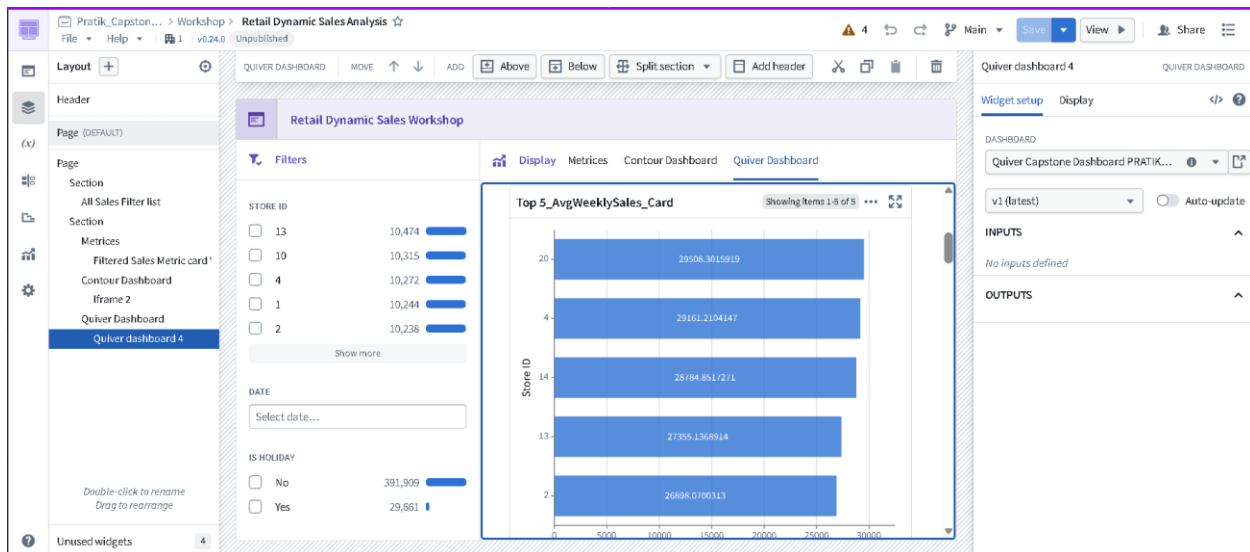
❖ Application using Workshop

Workshop was used to build the final interactive application for presenting the project insights. It brings together the analysis from Quiver and visualizations from Contour into a user-friendly interface with interactive filters and key performance metrics.

Steps:

1. Create a new Workshop application.
2. Design the application layout for retail sales analysis
3. Embed the Contour dashboard using an IFRAME widget.
4. Add interactive filters for Store ID, Date Range, and IsHoliday.
5. Display key metrics such as Total Sales and Average Weekly Sales.
6. Display the Top Performing Store
7. Connect the filters with the dashboard components.
8. Test the application to ensure the visualizations and filters work correctly.

- Build an interactive Workshop app to present insights from Quiver and visualizations from Contour.



➤ Embed the Contour dashboard to allow dynamic exploration of sales data.

I used IFRAME widget to embed Contour dashboard.

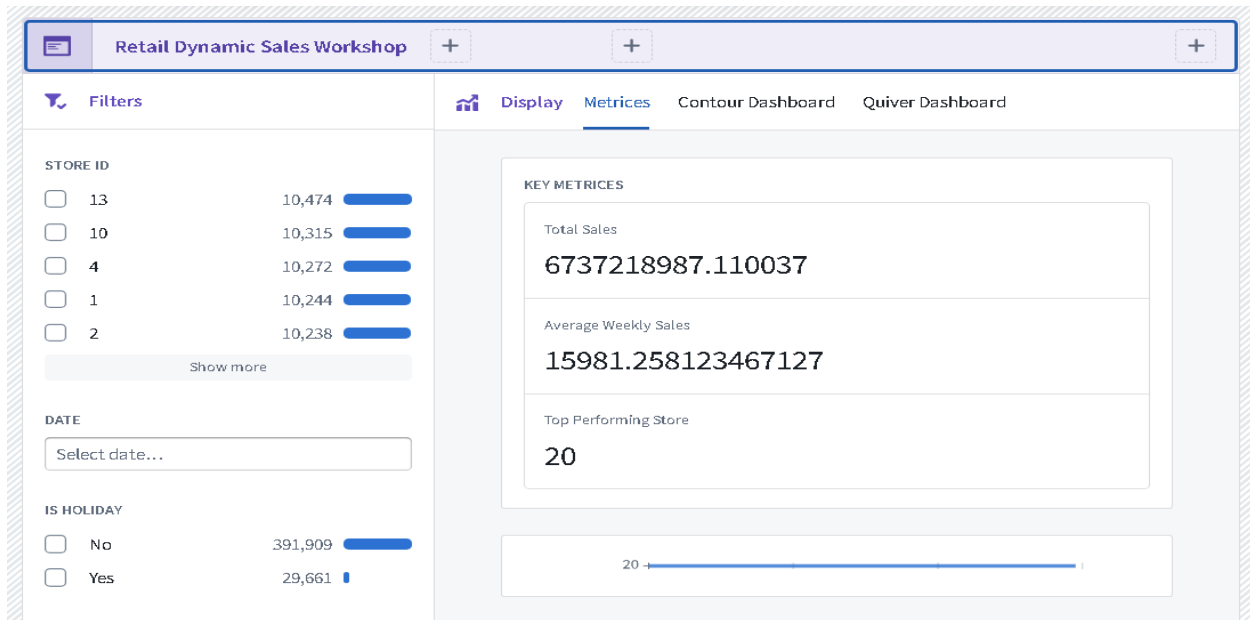
The screenshot shows the Palantir Foundry interface. On the left is a sidebar with a 'Filters' section containing 'STORE ID' (with a table of values), 'DATE' (with a date picker), and 'IS HOLIDAY' (with radio buttons). The main area displays the 'Contour Dashboard' which includes a line chart titled 'weekly_sales_trends_for_each_store'. On the right, the 'iframe 2' widget configuration is visible, showing the URL 'https://accenture.palantirfoundry.com/shares/lin...' and various settings like 'EXTERNAL FOUNDRY INSTANCE AUTHENTICATION' and 'ACCESSIBILITY LABEL'.

➤ Add filters for Store ID, Date Range, and IsHoliday for interactive analysis.

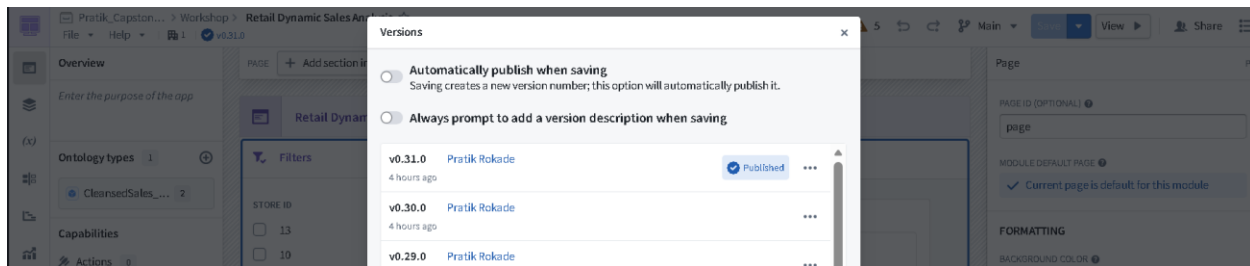
This close-up shows the 'Filters' section. The 'STORE ID' filter is active, showing a table with columns for 'STORE ID', 'Keeping values', and a bar chart. The 'DATE' filter has a 'Select date...' input. The 'IS HOLIDAY' filter has 'No' and 'Yes' radio buttons with corresponding counts and bar charts.

This close-up shows the 'INPUT DATA' and 'FILTERS CONFIGURATION' sections. The 'INPUT DATA' section shows 'OBJECT SET' as 'All Sales' and 'Current value' as '421,570'. The 'FILTERS CONFIGURATION' section shows a 'FILTER LIST' with 'Store ID', 'Date', and 'Is Holiday' filters, and a '+ Add filter' button.

- Display key metrics such as Total Sales, Average Weekly Sales, and Top Performing Store.



Published -



Key Outcomes –

- ✓ Clean & Reliable Data
- ✓ Automated Data Pipeline
- ✓ Business-Ready Ontology
- ✓ Actionable Sales Insights
- ✓ Interactive Contour Dashboard
- ✓ Workshop Business Application